ATELIER 9 Charger Bordeaux

Algorithmie comparative - J4 - Jeudi 12 novembre 2026, 11h25 - 75 minutes, en binôme

La question à laquelle l'atelier répond

Pour recalculer les tournées sur les vraies rues, il faut le réseau routier de la métropole en mémoire, dans le processus qui sert aussi le site et l'API. Comment le ranger ?

Ce que vous produisez : adr/GRILLE-ADR-006.md et adr/ADR-006-graphe.md (carnet), deux CSV dans bench/resultats.

Le rendu attendu

En fin d'atelier : le code, les mesures et la grille passent le correcteur, et l'ADR existe en premier jet (chaque rubrique en une à trois lignes, avec vos chiffres et le nom de vos CSV). Le correcteur sans KO avant 17h00, le commit du binôme à 17h00. Ensuite, l'ADR s'améliore chez vous jusqu'au dépôt du carnet, le 1er décembre.

Avant 17h00, le correcteur de l'atelier répond sans KO :

```
OK  À COMPLÉTER : les listes d'adjacence parcourent le graphe comme la matrice
OK  La matrice est mesurée sur des sous-graphes croissants
OK  Les listes sont mesurées à l'échelle de la métropole entière
OK  La grille comparative de l'ADR-006 est remplie
OK  L'ADR-006 a ses six rubriques, remplies
OK  La rubrique 3 de l'ADR-006 renvoie à une mesure qui existe, la rubrique 5 donne le domaine de validité
OK  Les trois phrases de l'ADR-006 tiennent sans jargon
7 contrôles, 0 KO
```

Avant de commencer. Le réseau est dans data/referentiel/graphe (OpenStreetMap, licence ODbL), arrivé avec maj-J4.zip. Créez l'ADR-006 du binôme, que vous remplissez dès le sous-objectif 1 :

Windows	`Copy-Item adr\ADR-000-gabarit.md adr\ADR-006-graphe.md`
macOS et Linux	`cp adr/ADR-000-gabarit.md adr/ADR-006-graphe.md`

Pour chaque sous-objectif, la première commande et ce que vous devez voir ; les suivantes sont dans le README du dépôt (tableau des commandes de la journée 4) et dans les fiches atelier numéro 5 à 7, celles de mardi.

Les sous-objectifs

1. Découvrir le réseau routier consigne 1 de la slide

Pourquoi. Savoir combien de carrefours et de rues on range, puis ce que coûterait la matrice avant de la construire.

Première action. Lancez la description du réseau, puis relevez en rubrique 1 de l'ADR-006 ses carrefours, ses arêtes et ce que coûterait la matrice.

Les trois systèmes	`npm run graphe`

Vous devez voir. Les mêmes nombres sur tous les postes : retenez la mémoire de la matrice et la part de ses cases utiles.

```
[graphe] 46 565 noeuds (intersections), 111 288 arêtes orientées, 5 177 km de voies environ
[graphe] une matrice d'adjacence de toute la métropole : 46 565 x 46 565 = 2,17 milliards de cases, soit
16,2 Go à 8 octets la case
[graphe] dont 0,0051 % de cases utiles : une case sur 19 484 contient une rue
```

2. Compléter les listes d'adjacence, lancer les tests consigne 2 de la slide

Pourquoi. Les listes ne gardent que les rues qui existent : c'est le seul À COMPLÉTER de l'atelier, deux ou trois lignes.

Première action. Complétez la variante listes de bench/variantes/representations.ts : rangez chaque arête dans la liste de son carrefour de départ, puis parcourez voisins[u]. Lancez npm test.

Windows	`code bench\variantes\representations.ts`
macOS et Linux	`code bench/variantes/representations.ts`

Vous devez voir. Les deux tests du réseau passent. Un seul test du dépôt échoue encore : celui de Dijkstra avec tas, qui attend l'atelier 10.

```
✔ matrice, listes et tableaux compacts parcourent le même sous-graphe (140.7707ms)
✔ un graphe d'un seul noeud se parcourt (0.1901ms)
```

3. Mesurer la matrice sur des sous-graphes croissants consigne 3 de la slide

Pourquoi. Un sous-graphe est une partie du réseau : les n premiers carrefours atteints depuis le hub de Brienne. La matrice y tient encore, sa croissance se voit.

Première action. Mesurez la matrice et les listes de 1 000 à 5 000 carrefours. Divisez la mémoire de la matrice à 5 000 par celle à 1 000 : ce rapport va dans la rubrique 3 de l'ADR-006.

Les trois systèmes

```
npm run bench -- --sujet graphe --variantes matrice,listes --tailles 1000,2000,5000 --repetitions 3 --echauffement 2
```

Vous devez voir. La mémoire de la matrice grandit comme le carré du nombre de carrefours ; celle des listes, non. Vos durées seront différentes, les mémoires à peine.

```
variante        taille   duree_ms   memoire_ko
matrice           1000       2.77         8124
matrice           2000       9.26        31598
matrice           5000      73.83       196073
```

4. Mesurer les listes à l'échelle de la métropole consigne 4 de la slide

Pourquoi. La décision porte sur la métropole entière, 46 565 carrefours : seules les listes et les tableaux compacts y tiennent.

Première action. Mesurez les listes et les tableaux compacts jusqu'aux 46 565 carrefours. Ne lancez pas la matrice à cette taille : Node s'arrête à court de mémoire.

Les trois systèmes

```
npm run bench -- --sujet graphe --variantes listes,compacts --tailles 5000,20000,46565 --repetitions 5 --echauffement 5
```

Vous devez voir. Toute la métropole en quelques millisecondes et quelques Mo. Comparez les deux : le temps et la mémoire.

```
variante        taille   duree_ms   memoire_ko
listes           46565       6.07        17813
compacts         46565       2.30         6808
```

5. Remplir la grille, écrire l'ADR-006, vérifier consignes 5, 6 de la slide

Pourquoi. Les poids de l'atelier (T-0733, son préalable) : temps 2, mémoire 4, lisibilité 2, robustesse 1, coût 1. Pour la métropole entière, mémoire : 5 sous 10 Mo, 4 sous 100 Mo, 3 sous 1 Go, 2 sous 4 Go, 1 au-delà ; temps : 5 sous 10 ms, 4 sous 100 ms, 3 sous 1 s, 2 sous 10 s, 1 au-delà ou impossible. La lisibilité, la robustesse et le coût ont chacun leur barème, en annexe.

Première action. Copiez la grille, notez les trois options, testez la sensibilité des poids, puis écrivez l'ADR-006 : sa rubrique 5 donne le domaine de validité. En 2027, avec Libourne et le Bassin, Efrei prévoit 12 000 colis par jour ouvré, contre 6 000 en septembre 2026 (pièce 06 de la data room, sur Moodle depuis le 1er octobre). Préparez la décision et la question de compréhension à deux, puis lancez `npm run verifier -- 9`.

Windows

```
Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-006.md
```

macOS et Linux

```
cp adr/grille-comparative.md adr/GRILLE-ADR-006.md
```

- En atelier, l'ADR est un premier jet ; ce qui manque se reprend cet après-midi, dans le créneau de l'atelier 10, puis vous la finissez chez vous, pour le carnet du 1er décembre.
- Rubrique 3 : remplacez les colonnes du tableau du gabarit par 1 000, 5 000, 20 000 et 46 565 carrefours, puis la mémoire à 46 565.
- Rubrique 6 : ni unité (ms, Mo), ni nom de méthode ou de structure ; le correcteur donne d'un coup tous les mots qui coincent.

Vous devez voir. Le correcteur répond sans KO, comme en première page. Il vérifie les listes, les deux mesures, la grille et l'ADR.

La décision de l'atelier

Quelle façon de ranger le réseau retenez-vous, et qu'est-ce qui interdit la matrice, avec la mémoire mesurée ou calculée ? Entre listes et tableaux compacts, la décision est fragile : la sensibilité le montre.

La question de compréhension

À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : en fin d'atelier, l'intervenant passe la demander à quelques binômes.

À 17h00, le binôme commite : git add -A puis git commit -m "J4". Sans commit, le travail du jour n'est pas évalué.

Annexe, à consulter au besoin

Les cas particuliers

Le réseau est absent (npm run graphe le dit) : maj-J4.zip n'est pas décompressé à la racine du dépôt :

Windows	`Expand-Archive -Path $HOME\Downloads\maj-J4.zip -DestinationPath . -Force`
macOS et Linux	`unzip -o ~/Downloads/maj-J4.zip -d .`

Node s'arrête avec "JavaScript heap out of memory" : la matrice a été lancée sur la métropole entière. Relancez la mesure du sous-objectif 4 sans elle : `--variantes listes,compacts`.

Les barèmes des critères sans chronomètre

- **Mémoire** (poids 4) : la colonne memoire_ko du harness à 46 565 carrefours, lue comme un ordre de grandeur ; deux options à moins d'un facteur 2 l'une de l'autre ont la même note. La matrice ne se mesure pas à cette taille : sa mémoire se calcule (npm run graphe), et la rubrique 3 doit le dire.
- **Lisibilité** (poids 2) : lisez les trois variantes de bench/variantes/representations.ts, écrites avant vous, et répondez pour chacune à trois questions : que fait la variante, comment lit-elle les voisins d'un carrefour, où retirer une rue. Listez les idées qu'il faut connaître pour répondre ; on part de 5 et chaque idée (un tableau de listes, le tableau debut qui indexe les deux autres, la lecture de debut[u] à debut[u + 1] - 1) coûte un point.
- **Robustesse** (poids 1) : `npm test` d'abord, puis ce que la représentation garde du réseau. Barème : 5 si elle garde toutes les arêtes, y compris deux tronçons parallèles entre les mêmes carrefours (une rue et sa contre-allée) ; 3 si elle en perd mais le dit ; 2 si elle en perd en silence ; 1 si `npm test` échoue. Indice : combien de cases la matrice a-t-elle pour une paire de carrefours ?
- **Coût de mise en oeuvre** (poids 1) : les lignes à écrire et ce qu'il faut refaire quand le réseau change (un pont qui se lève). Barème : 5 si rien n'est à écrire ; 4 pour quelques lignes sans donnée à reconstruire ; 3 pour quelques lignes et une donnée à reconstruire quand le réseau change ; 2 pour une structure nouvelle dans l'équipe ; 1 si une dépendance s'ajoute.
- Dans tous les cas, justifiez la note en une phrase sous la grille : le barème dit la note, la phrase dit ce que vous avez observé.

Où trouver un exemple dans le code

bench/variantes/representations.ts contient la matrice et les tableaux compacts, écrits en entier : la matrice montre comment une variante construit puis parcourt ; les tableaux compacts, comment on lit les rues d'un carrefour. bench/graphe/graphe.ts lit le réseau et fabrique les sous-graphes.

Si ça coince

- npm run graphe répond que le réseau est absent : maj-J4.zip n'est pas décompressé à la racine du dépôt (annexe).
- Le test "matrice, listes et tableaux compacts parcourent le même sous-graphe" échoue : les arêtes sont rangées dans la liste du carrefour d'arrivée, ou le parcours ne lit pas voisins[u].
- Node s'arrête à court de mémoire : la matrice a été lancée sur la métropole entière ; mesurez-la jusqu'à 5 000 carrefours seulement.

Toujours bloqué après ces pistes : levez la main, ou écrivez sur Teams.

Si vous avez fini

- Calculez la mémoire de la matrice à 15 000, puis à 100 000 carrefours, et la taille où elle dépasse le tas de Node (2 à 4 Go) : écrivez-la en rubrique 5.
- Calculez combien de carrefours il faudrait pour que les tableaux compacts dépassent 100 Mo, puis comparez avec Libourne et le Bassin (pièce 06).

Les sorties montrées sont de vraies sorties, capturées en jouant la fiche atelier en cours ; vos durées seront différentes, vos nombres de carrefours, d'adresses retrouvées, de tournées et vos distances non.